

EL COMANDO PRINT EN PYTHON

Documento de Referencia Completo

Versión 1.0

Resumen: Explicación detallada del comando print(), su sintaxis, parámetros y ejemplos prácticos.

3 de septiembre de 2026

Autor: Documentación Técnica

1. Introducción

El comando `print()` es una de las funciones más fundamentales y utilizadas en Python. Su propósito principal es mostrar información en la consola o terminal, permitiendo al programador visualizar resultados, depurar código y comunicar mensajes al usuario.

1.1. Sintaxis Básica

La sintaxis más simple del comando es:

```
1 print("Hola Mundo")
```

2. Parámetros del Comando print()

El comando `print()` acepta varios parámetros que modifican su comportamiento:

2.1. Parámetro sep

Define el separador entre múltiples argumentos. Por defecto es un espacio.

```
1 print("Python", "es", "genial", sep="-")
2 # Salida: Python-es-genial
```

2.2. Parámetro end

Define qué carácter se añade al final de la salida. Por defecto es un salto de línea (`\n`).

```
1 print("Hola", end=" ")
2 print("Mundo")
3 # Salida: Hola Mundo
```

2.3. Parámetro file

Permite redirigir la salida a un archivo en lugar de la consola.

```
1 with open('salida.txt', 'w') as f:
2     print("Texto en archivo", file=f)
```

3. Ejemplos Prácticos

3.1. Imprimir Variables

```
1 nombre = "Ana"
2 edad = 25
3 print("Nombre:", nombre, "Edad:", edad)
4 # Salida: Nombre: Ana Edad: 25
```

3.2. Formateo de Cadenas

```
1 # f-strings (Python 3.6+)
2 print(f"Mi nombre es {nombre} y tengo {edad} a os")
3
4 # M todo format()
5 print("Mi nombre es {} y tengo {} a os".format(nombre, edad))
```

3.3. Imprimir Caracteres Especiales

```
1 print("Línea 1\nLínea 2") # Salto de línea
2 print("Tabulador\tTexto") # Tabulador
3 print("Comilla simple: '") # Comillas simples
4 print("Comilla doble: \") # Comillas dobles
```

4. Errores Comunes

- Olvidar los paréntesis: `print "Hola"` (incorrecto en Python 3)
- No cerrar las comillas: `print("Hola)` (error de sintaxis)
- Mezclar tipos de datos sin convertirlos: `print(.5 número es 5)` (error)

5. Conclusión

El comando `print()` es esencial en Python. Dominar sus parámetros y opciones de formato permite crear salidas claras y profesionales, facilitando el desarrollo y la depuración de programas.

OPERACIONES MATEMÁTICAS EN PYTHON

Guía Completa de Operadores Aritméticos

Versión 1.0

Resumen: Explicación de todos los operadores matemáticos básicos, precedencia, y ejemplos prácticos.

3 de septiembre de 2026

Autor: Documentación Técnica

6. Introducción

Python ofrece un conjunto completo de operadores matemáticos que permiten realizar cálculos de todo tipo, desde operaciones básicas hasta cálculos científicos complejos.

7. Operadores Aritméticos Básicos

7.1. Suma (+) y Resta (-)

```
1 a = 10
2 b = 3
3 print(a + b) # 13
4 print(a - b) # 7
```

7.2. Multiplicación (*) y División (/)

```
1 print(a * b) # 30
2 print(a / b) # 3.3333333333333335 (Siempre devuelve float)
```

7.3. División Entera (//) y Módulo (%)

```
1 print(a // b) # 3 (Parte entera de la división)
2 print(a % b) # 1 (Resto de la división)
```

7.4. Potenciación (**)

```
1 print(2 ** 3) # 8 (2 elevado a la 3)
2 print(4 ** 0.5) # 2.0 (Raíz cuadrada)
```

8. Precedencia de Operadores

El orden de evaluación es:

1. Paréntesis: ()
2. Potenciación: **
3. Multiplicación, división, división entera y módulo: *, /, //, %
4. Suma y resta: +, -

```
1 resultado = 2 + 3 * 4 ** 2 # 2 + 3 * 16 = 2 + 48 = 50
2 print(resultado) # 50
3
4 resultado = (2 + 3) * 4 ** 2 # 5 * 16 = 80
5 print(resultado) # 80
```

9. Operadores de Asignación Compuestos

```
1 x = 5
2 x += 3 # Equivalente a x = x + 3
3 print(x) # 8
4
5 x *= 2 # Equivalente a x = x * 2
6 print(x) # 16
7
8 x //= 3 # Equivalente a x = x // 3
9 print(x) # 5
```

10. Funciones Matemáticas Incorporadas

Python incluye funciones matemáticas básicas sin necesidad de importar módulos:

```
1 print(abs(-5)) # 5 (Valor absoluto)
2 print(max(3, 7, 2)) # 7 (Máximo)
3 print(min(3, 7, 2)) # 2 (Mínimo)
4 print(round(3.14159, 2)) # 3.14 (Redondeo)
5 print(pow(2, 3)) # 8 (Potencia, equivalente a 2**3)
```

11. Módulo math

Para operaciones avanzadas:

```
1 import math
2
3 print(math.sqrt(16)) # 4.0 (Raíz cuadrada)
4 print(math.sin(math.pi/2)) # 1.0 (Seno)
5 print(math.log(100, 10)) # 2.0 (Logaritmo base 10)
6 print(math.factorial(5)) # 120 (Factorial)
```

12. Ejemplos Prácticos

12.1. Cálculo de Área de un Círculo

```
1 import math
2
3 radio = 5
4 area = math.pi * radio ** 2
5 print(f" rea del c rculo: {area:.2f}") # 78.54
```

12.2. Conversión de Temperatura

```
1 celsius = 25
2 fahrenheit = (celsius * 9/5) + 32
3 print(f"{celsius} C = {fahrenheit} F ") # 25 C = 77.0 F
```

13. Errores Comunes

- División por cero: $10 / 0$ (Error: ZeroDivisionError)
- Confundir división entera con división normal
- No considerar la precedencia de operadores

14. Conclusión

Los operadores matemáticos en Python son potentes y flexibles. Conocer su funcionamiento y precedencia es crucial para escribir código correcto y eficiente.

FUNCIONES EN PYTHON (def)

Definición, Parámetros, Retorno y Ámbito

Versión 1.0

Resumen: Guía completa sobre la definición y uso de funciones en Python con la palabra clave **def**.

3 de septiembre de 2026

Autor: Documentación Técnica

15. Introducción

Las funciones son bloques de código reutilizables que realizan una tarea específica. En Python, se definen usando la palabra clave `def`. Las funciones permiten organizar el código, evitar repeticiones y facilitar el mantenimiento.

16. Sintaxis Básica

```
1 def nombre_funcion(parametros):  
2     """Docstring: Explicación de la función"""  
3     # Código de la función  
4     return valor # Opcional
```

16.1. Ejemplo Simple

```
1 def saludar():  
2     print(" ¡Hola !")  
3  
4 saludar() # Llama a la función  
5 # Salida: ¡Hola !
```

17. Parámetros y Argumentos

17.1. Parámetros Obligatorios

```
1 def sumar(a, b):  
2     return a + b  
3  
4 print(sumar(5, 3)) # 8
```

17.2. Parámetros con Valores por Defecto

```
1 def saludar_persona(nombre, saludo="Hola"):  
2     print(f"{saludo}, {nombre}")  
3  
4 saludar_persona("Ana") # Hola, Ana  
5 saludar_persona("Ana", "Buenos días") # Buenos días, Ana
```

17.3. Parámetros Arbitrarios (*args y **kwargs)

```
1 def sumar_todos(*args):
2     return sum(args)
3
4 print(sumar_todos(1, 2, 3, 4)) # 10
5
6 def mostrar_info(**kwargs):
7     for clave, valor in kwargs.items():
8         print(f"{clave}: {valor}")
9
10 mostrar_info(nombre="Ana", edad=25, ciudad="Madrid")
```

18. Retorno de Valores

18.1. Retorno Simple

```
1 def cuadrado(n):
2     return n ** 2
3
4 print(cuadrado(4)) # 16
```

18.2. Retorno Múltiple

```
1 def operaciones(a, b):
2     return a + b, a - b, a * b
3
4 suma, resta, multi = operaciones(10, 3)
5 print(suma, resta, multi) # 13 7 30
```

18.3. Función sin Retorno

```
1 def mostrar_mensaje():
2     print("Esta funci n no retorna nada")
3
4 resultado = mostrar_mensaje()
5 print(resultado) # None
```

19. Ámbito de Variables

19.1. Variables Locales

```
1 def mi_funcion():
2     x = 10 # Variable local
3     print(x)
4
5 mi_funcion() # 10
6 # print(x) # Error: NameError (x no existe fuera)
```

19.2. Variables Globales

```
1 x = 5 # Variable global
2
3 def modificar():
4     global x
5     x = 10
6
7 modificar()
8 print(x) # 10
```

20. Documentación de Funciones (Docstrings)

```
1 def calcular_promedio(numeros):
2     """
3     Calcula el promedio de una lista de n meros.
4
5     Args:
6         numeros (list): Lista de n meros.
7
8     Returns:
9         float: Promedio de los n meros.
10    """
11    return sum(numeros) / len(numeros)
12
13 print(calcular_promedio.__doc__) # Muestra la documentaci n
14 help(calcular_promedio) # Muestra ayuda formateada
```

21. Funciones Anidadas

```
1 def exterior(x):
2     def interior(y):
3         return x + y
4     return interior
5
```

```
6 suma_con_5 = exterior(5)
7 print(suma_con_5(3)) # 8 (5 + 3)
```

22. Ejemplos Prácticos

22.1. Calculadora Simple

```
1 def calculadora(a, b, operacion):
2     if operacion == "suma":
3         return a + b
4     elif operacion == "resta":
5         return a - b
6     elif operacion == "multiplicacion":
7         return a * b
8     elif operacion == "division":
9         return a / b
10    else:
11        return "Operación no válida"
12
13 print(calculadora(10, 5, "suma")) # 15
```

22.2. Validación de Datos

```
1 def validar_edad(edad):
2     if edad < 0:
3         return False
4     return True
5
6 def ingresar_edad():
7     edad = int(input("Ingresa tu edad: "))
8     if validar_edad(edad):
9         print(f"Edad válida: {edad}")
10    else:
11        print("Edad inválida")
12
13 # ingresar_edad() # Descomentar para probar
```

23. Errores Comunes

- Olvidar los dos puntos al final de la definición
- No respetar la indentación (el cuerpo de la función debe estar indentado)

- Usar nombres de funciones que coinciden con palabras reservadas
- No retornar valores cuando se espera un resultado

24. Conclusión

Las funciones `def` son fundamentales en Python. Permiten escribir código modular, reutilizable y fácil de mantener. Dominar su uso es esencial para cualquier programador.